# PSC INF : RAPPORT FINAL

## Machine Learning Quantique dans la Détection de Fraude Bancaire

Avril 2026

Hannah Faucheu, Guillaume Hentotte, Achim Kobena Dongo, Tancrède Tonnini, Kristen Boitier, Stanislas Mischler

# TABLE DES MATIÈRES


**1 Introduction** **2**
1.1 Motivation générale . . . 3
1.2 État de l'art . . . 3
1.2.1 Approches classiques de référence . . . 3
1.2.2 Clustering quantique . . . 4
1.2.3 Détection d'anomalies quantique . . . 4
1.2.4 Classifieurs QML supervisés . . . 4

**2 Présentation du projet** **5**
2.1 Méthode générale . . . 6
2.2 Données utilisées . . . 6

**3 Données de la BNP** **7**

**4 Techniques de preprocessing** **7**

**5 Clustering** **8**
5.1 Clustering classique . . . 8
5.2 Clustering quantique variationnel . . . 8
5.3 Implémentation . . . 11
5.4 Rappels théoriques : SVM classique et astuce du noyau . . . 12
5.5 Quantum One-Class Support Vector Machine (Q-OCSVM) . . . 13

**6 Réseaux à couches** **15**

**7 Assemblage d'une pipeline** **15**
7.1 Architecture modulaire . . . 15
7.2 Résultats . . . 15

**8 Tests et Comparaisons** **15**
8.1 Comparaison de 4 algorithmes sur un dataset tiers . . . 15
8.2 Application au jeu de données de la BNP . . . 15

**9 Difficultés et critique** **16**
9.1 Limites matérielles . . . 16
9.2 Absence de labels dans le jeu de données de la BNP . . . 16

**10 Conclusion et idées futures** **16**
10.1 Bilan . . . 16
10.2 Idées futures . . . 16


# 1 INTRODUCTION

La détection de fraude bancaire est aujourd'hui l'un des enjeux opérationnels majeurs des grandes institutions financières. Le volume toujours croissant des transactions, la diversification des canaux de paiement et l'adaptation permanente des comportements frauduleux rendent la tâche particulièrement difficile pour les approches

classiques, qui doivent composer avec des jeux de données massifs, déséquilibrés et en constante évolution. Dans ce contexte, l'informatique quantique, et plus particulièrement le *Quantum Machine Learning* (QML), apparaît comme une voie prometteuse pour dépasser certaines limites théoriques des algorithmes classiques.

Ce rapport final s'inscrit dans la continuité de nos travaux menés en collaboration avec BNP Paribas dans le cadre du PSC INF. Il présente la synthèse de notre démarche, depuis la définition du problème jusqu'à l'assemblage d'une pipeline hybride classique/quantique de détection de fraude, en passant par l'étude des briques algorithmiques individuelles — clustering, réseaux de neurones, détection d'anomalies — et par leur confrontation à des données réelles fournies par la banque.

## 1.1 Motivation générale

La détection de fraude constitue un enjeu à la fois économique, réglementaire et technique pour les institutions bancaires. Chaque transaction frauduleuse non détectée se traduit par une perte financière directe, une atteinte à la confiance des clients et une exposition accrue aux contraintes de conformité. À l'inverse, un taux trop élevé de faux positifs dégrade l'expérience utilisateur et mobilise inutilement les équipes d'analyse. Les méthodes classiques de *machine learning*, bien que matures, atteignent progressivement leurs limites face à la croissance du volume et de la complexité des données financières, ainsi qu'à la nature intrinsèquement mouvante de la fraude.

Les algorithmes quantiques promettent, du moins sur le plan théorique, des gains significatifs en complexité et en expressivité par rapport à leurs équivalents classiques, notamment pour des tâches de clustering, de détection d'anomalies ou de recherche dans des espaces de grande dimension. Cette promesse reste toutefois largement conditionnée par la maturité du matériel actuel — processeurs NISQ bruités, nombre limité de qubits, absence de QRAM opérationnelle — et par la capacité à concevoir des algorithmes véritablement adaptés aux contraintes du domaine bancaire.

Notre projet vise précisément à explorer ce compromis entre potentiel théorique et faisabilité pratique. En collaboration avec BNP Paribas, nous avons cherché à concevoir et à implémenter une pipeline de détection de fraude intégrant des briques quantiques à des composantes classiques éprouvées, de manière à pouvoir mesurer rigoureusement l'apport de chaque approche. L'objectif final est double : d'une part, fournir à la BNP un proof of concept d'algorithme hybride capable d'analyser des transactions passées pour signaler les fraudes éventuelles sur de nouvelles transactions ; d'autre part, identifier les briques pour lesquelles l'approche quantique offre, dès aujourd'hui ou à moyen terme, un avantage tangible sur ses contreparties classiques.

## 1.2 État de l'art

La détection de fraude est un problème classique de classification supervisée fortement déséquilibrée, pour lequel la littérature classique est abondante (régressions logistiques, gradient boosting, forêts aléatoires, auto-encodeurs, réseaux de neurones profonds). Notre objectif n'est pas de rivaliser frontalement avec ces méthodes, mais d'examiner dans quelle mesure des algorithmes quantiques, ou hybrides classique/quantique, peuvent apporter un gain *théorique* (complexité, expressivité) ou *pratique* (qualité de détection) sur ce type de données. Nous distinguons quatre familles de travaux qui structurent notre état de l'art : les approches classiques de référence, le clustering quantique, la détection d'anomalies quantique, et les classifieurs QML supervisés.

### 1.2.1 • Approches classiques de référence

Les méthodes classiques déployées en production reposent essentiellement sur des modèles supervisés entraînés sur des historiques étiquetés. Le gradient boosting (XGBoost, LightGBM) et les forêts aléatoires constituent aujourd'hui les *baselines* usuelles, car ils gèrent bien les variables hétérogènes et offrent une bonne robustesse au bruit. Pour les jeux de données fortement déséquilibrés comme ceux de la fraude (où la classe positive représente typiquement moins de 0,2% des transactions), l'emploi de techniques de rééchantillonnage telles que SMOTE [2] et de prétraitements par ACP est devenu standard. En parallèle, les méthodes non supervisées — Isolation

Forest, Local Outlier Factor, auto-encodeurs — sont utilisées soit pour pallier l'absence d'étiquettes fiables, soit pour construire des scores d'anomalie utilisés en amont d'un classifieur supervisé. Ces approches constituent le point de comparaison incontournable pour tout algorithme quantique prétendant améliorer la détection de fraude.

### 1.2.2 • Clustering quantique

Le clustering quantique a émergé avec la popularisation du *Quantum Machine Learning* (QML) [3]. Deux grandes familles se distinguent. La première est celle des algorithmes *théoriquement optimaux* supposant l'existence d'une QRAM : Kerenidis et al. proposent un algorithme de $q$-means [4] dont la complexité est polylogarithmique en la taille du jeu de données, ainsi qu'un clustering spectral quantique [5] exploitant l'estimation de phase pour diagonaliser efficacement la matrice laplacienne. Ces résultats sont spectaculaires sur le plan asymptotique, mais reposent sur des hypothèses d'accès aux données encore débattues, la réalisation physique d'une QRAM efficace restant un problème ouvert.

La seconde famille, *variationnelle*, est spécifiquement conçue pour les ordinateurs bruités de type NISQ (*Noisy Intermediate-Scale Quantum*). Bermejo et Orús [7] proposent ainsi un clustering variationnel où un circuit paramétré $U(\theta)$ est optimisé par une boucle classique pour minimiser une fonction de coût reflétant la séparation inter-clusters. Poggiali et al. [6] décrivent une variante hybride de $k$-means dans laquelle le calcul des distances est délégué à un sous-circuit quantique. Ces approches renoncent à la garantie de complexité polylogarithmique, mais sont implémentables sur le matériel actuel.

### 1.2.3 • Détection d'anomalies quantique

La détection d'anomalies quantique est un domaine plus jeune et plus fragile : plusieurs articles publiés entre 2019 et 2023 contiennent des erreurs importantes, comme l'illustre Guo et al. [8], qui réfutent l'algorithme de Liang et al. [9] et en proposent une version corrigée fondée sur des rotations contrôlées. La démarche générale reprend le schéma classique en deux temps : (i) apprentissage d'une représentation du comportement « normal » par clustering, et (ii) identification des outliers par une mesure de distance ou de densité. Les variantes quantiques diffèrent essentiellement par l'étape (ii), où l'amplitude d'un état produit ou le résultat d'une estimation d'amplitude fournissent un score d'anomalie. Là encore, la quasi-totalité des publications suppose un accès QRAM, ce qui limite leur portée expérimentale immédiate.

Une approche récente particulièrement adaptée aux contraintes NISQ est celle des *Quantum Autoencoders* (QAE), dont FiD-QAE [13] constitue une instance directement appliquée à la fraude bancaire. Le principe est simple et élégant : on encode chaque transaction $\vec{x} \in \mathbb{R}^N$ par *amplitude encoding* dans $n = \lceil \log_2 N \rceil$ qubits, puis un encodeur paramétré $U(\theta)$ partitionne le registre en un sous-espace *latent* $A$ (de dimension $n_A$) et un sous-espace *trash* $B$ (de dimension $n_B$). On entraîne ensuite $\theta$ *exclusivement sur les transactions légitimes* pour que le sous-espace $B$ soit comprimé sur un état de référence fixe $|0\rangle^{\otimes n_B}$, ce qui se mesure par la fidélité $F(\theta) = \langle 0|\rho_B(\theta)|0\rangle$ via un SWAP test. La fonction de coût minimisée est alors $\mathcal{L}(\theta) = 1 - \mathbb{E}_{\vec{x}}[F(\theta;\vec{x})]$. À l'inférence, une transaction frauduleuse — non vue à l'entraînement — produit une fidélité significativement plus basse, et un seuil $\tau$ permet de la signaler. L'élégance de cette construction réside dans son traitement implicite du déséquilibre de classes : on n'a jamais besoin d'exemples de fraude pour entraîner le modèle. Les auteurs rapportent une exactitude supérieure à 0,92 et une précision supérieure à 0,80 avec seulement quatre qubits, ce qui en fait un candidat sérieux pour notre brique de détection d'anomalies non supervisée.

### 1.2.4 • Classifieurs QML supervisés

Le troisième pan de la littérature concerne les classifieurs supervisés quantiques, principalement les *Variational Quantum Classifiers* (VQC) et les *Quantum Neural Networks* (QNN). Un VQC encode une donnée classique $\vec{x} \in \mathbb{R}^d$ dans un état quantique $|\psi(\vec{x})\rangle = U_{\varphi(\vec{x})}\,|0\rangle^{\otimes n}$ via un *feature map* $\varphi$, applique ensuite un ansatz paramétré $U(\theta)$, puis mesure une observable dont la valeur espérée définit un score de classification. Les paramètres $\theta$ sont optimisés classiquement (COBYLA, SPSA, Adam) pour minimiser une perte, typiquement la cross-entropie. L'introduction des *feature maps* de Pauli et en particulier du *ZZ feature map* par Havlíček et al. [10] a marqué

un jalon en établissant un cadre reproductible pour exploiter les corrélations d'ordre deux entre variables dans l'espace de Hilbert.

Plusieurs travaux récents enrichissent significativement ce paysage et nourrissent directement nos choix de conception. El Alami et al. [11] fournissent une analyse comparative récente et particulièrement pertinente pour notre projet. Ils évaluent trois architectures QML — VQC, *Sampler QNN* (SQNN, qui extrait des échantillons de la distribution de mesure pour alimenter un optimiseur classique) et *Estimator QNN* (EQNN, hybride à base d'estimation d'observables suivie d'un réseau de neurones classique) — en faisant varier trois feature maps (Pauli, ZZ, Z) et quatre ansätze (Real Amplitudes, Efficient SU2, Two Local, Pauli Two Design). Leurs expériences sont menées sur le jeu public *European credit card* (celui fourni dans `Data/creditcard.csv`) et sur le simulateur BankSim, après rééquilibrage *Random Under Sampling* à 50/50. Ils recourent à une analyse de variance (ANOVA) pour quantifier l'effet individuel et croisé des composants, ce qui constitue une contribution méthodologique utile : elle permet d'identifier les combinaisons feature map/ansatz statistiquement significatives, et de dégager des configurations dont l'exactitude dépasse 0,98 de F1-score sur leurs jeux de test. Ce travail nous servira de référence explicite dans la section 5, à la fois comme source d'inspiration méthodologique (le protocole d'évaluation systématique) et comme témoin des performances atteignables par des classifieurs QML purement supervisés — complémentaires de notre approche, qui privilégie la détection d'anomalies et le clustering non supervisé.

Là où El Alami et al. fixent manuellement les paires *feature map/ansatz* à comparer, Innan et al. proposent avec *CircuitHunt* [12] une démarche radicalement différente : ils *automatisent* la recherche de circuits adaptés à la détection de fraude. Le principe repose sur un dataset pré-construit de circuits quantiques (KetGPT) couvrant des largeurs de 3 à 10 qubits, sur lequel on applique trois filtres successifs : un budget en qubits, un nombre maximal de paramètres entraînables (typiquement $P_{\max} = 30$), et une vérification de validité numérique par simulation à paramètres nuls. Chaque circuit candidat est ensuite intégré dans une architecture hybride uniforme — pré-réseau classique, encodage du vecteur d'entrée dans le circuit candidat, connexion résiduelle apprenable, post-réseau classique — et entraîné brièvement (5 époques) ; le meilleur macro-F1 sur validation détermine le circuit retenu, qui est alors ré-entraîné plus longuement. Sur le jeu *Credit Card Fraud* (rééquilibré par SMOTE), le circuit sélectionné atteint un macro-F1 de 0,97 et une AUC de 0,9946, surpassant plusieurs architectures QML conçues à la main (EstimatorQNN, QGNN, SQNN, QFDNN). Cette méthodologie nous est doublement précieuse : d'une part comme *baseline* très exigeante à dépasser, d'autre part comme inspiration pour notre propre démarche d'évaluation systématique des briques quantiques.

Enfin, dans une direction orthogonale, Ubale et al. [14] explorent la composante *temporelle* des transactions en proposant un *Quantum LSTM* hybride. Leur architecture chaîne (i) un LSTM classique extrayant des dépendances temporelles d'une fenêtre glissante de transactions, (ii) une couche pleinement connectée projetant l'état caché $h_T$ vers $n_{\text{qubits}}$ dimensions, (iii) un VQC composé d'un *AngleEmbedding* ($R_y(x_i)\,|0\rangle$ par qubit) suivi de couches *StronglyEntanglingLayers* paramétrées, (iv) une mesure dans la base de Pauli-$Z$ produisant un vecteur de features quantiques, et (v) une couche dense finale de classification. Tout est entraîné de bout en bout par rétropropagation, les gradients du circuit étant calculés par la *parameter-shift rule*. Sur des données synthétiques de 50 000 transactions enrichies par des features géospatiales (distance haversine entre client et marchand) et temporelles, leur QLSTM dépasse une LSTM classique sur précision, rappel et F1, tout en restant entraînable sur CPU en environ une minute par époque. Cet article suggère qu'une partie de l'avantage potentiel du quantique en détection de fraude pourrait précisément résider dans le traitement de données séquentielles, perspective que nous garderons en tête si les jeux BNP comportent une structure temporelle exploitable.

# 2
# PRÉSENTATION DU PROJET

## 2.1 Méthode générale

Notre démarche reprend et affine le squelette adopté par le groupe 2024-25 : une pipeline de détection de fraude articulée autour d'un bloc de *clustering* — pour apprendre une représentation du comportement normal — et d'un bloc de *détection d'anomalies* — pour signaler les transactions s'en écartant —, le tout assemblé dans une architecture modulaire dite *plug & play*. Notre contribution principale sur ce squelette est d'en pousser la décomposition : là où nos prédécesseurs traitaient le clustering et la détection d'anomalies comme deux étapes relativement couplées, nous cherchons à isoler plus finement les sous-étapes (préparation des données, extraction de features, apprentissage du modèle de normalité, calcul du score d'anomalie, seuillage de décision) afin d'autoriser une hybridation plus souple entre composantes classiques et quantiques. Chaque sous-étape respecte une interface commune et peut être remplacée par sa contrepartie alternative sans modifier le reste de la pipeline, ce qui nous permet de composer et de comparer des configurations entièrement classiques, entièrement quantiques, ou arbitrairement mixtes.

Les briques quantiques effectivement implémentées dans ce rapport sont concentrées autour du *clustering quantique variationnel*, qui constitue le cœur algorithmique de notre contribution. D'autres pistes quantiques — en particulier au niveau du préprocessing (réduction de dimension, encodage, variantes variationnelles de détection d'anomalies) — ont été explorées de manière plus prospective et font l'objet de discussions qualitatives plutôt que d'un benchmark exhaustif. Les architectures QML supervisées évoquées dans l'état de l'art (VQC, QAE, QLSTM) servent quant à elles de points de comparaison bibliographiques et n'ont pas été réimplémentées dans le cadre de ce projet.

Notre travail s'est déroulé en trois phases. Dans un premier temps, l'ensemble du groupe s'est consacré à une étude collective de la littérature QML et à la prise en main des outils d'informatique quantique (Qiskit, simulateurs, primitives variationnelles). Dans un deuxième temps, les différentes briques ont été développées en parallèle, chacune faisant l'objet d'un traitement théorique et expérimental autonome. Le dernier mois a enfin été consacré à l'intégration de ces briques dans l'architecture modulaire décrite ci-dessus, et à la mise en place de benchmarks comparatifs classiques / quantiques sur un jeu de données commun.

À ce stade du projet, nous ne visons pas d'exécution sur processeur quantique réel : toutes nos expériences sont menées sur `AerSimulator`, ce qui nous autorise des tailles d'instance et des budgets de *shots* supérieurs à ceux accessibles via les services cloud d'IBM, au prix de l'absence de bruit matériel. Nous ne faisons par ailleurs pas d'hypothèses simplificatrices fortes sur la distribution des données : contrairement au proof of concept de 2024-25, qui supposait des clusters gaussiens dans $\mathbb{R}^d$, notre pipeline est conçue pour s'accommoder de features hétérogènes et de distributions non paramétriques. Les limites matérielles et leurs conséquences sur la portée de nos conclusions sont discutées en section 5 et dans le chapitre consacré aux difficultés et critiques.

## 2.2 Données utilisées

Nous avons travaillé sur deux jeux de données complémentaires. Le premier, fourni par BNP Paribas, est le jeu cible de notre étude ; sa particularité est de comporter des variables catégorielles à forte cardinalité — identifiants de traders, localisations — qui ne se prêtent pas directement aux algorithmes géométriques sur lesquels reposent la plupart de nos briques de clustering et de détection d'anomalies. Nous consacrons une section dédiée (3) aux pistes explorées pour transformer ces features qualitatives en représentations numériques exploitables. Le second jeu, utilisé pour le benchmarking systématique de nos briques, est un jeu public de transactions Revolut publié sur Kaggle [16] : directement étiqueté en classes fraude / non-fraude, il fournit une vérité terrain nette qui nous permet de comparer rigoureusement les variantes classiques et quantiques de chaque brique, et d'isoler ainsi l'apport algorithmique indépendamment des difficultés de préparation propres aux données BNP.

# 3
# DONNÉES DE LA BNP

# 4
# TECHNIQUES DE PREPROCESSING

Le but du machine learning est de faire de l'apprentissage à partir de notre jeu de données. Cependant, une étape clé en amont de l'apprentissage est de modifier ce jeu de données pour le rendre plus exploitable, plus performant, plus apte à révéler de l'information. Ainsi nous présentons plusieurs méthodes dites de Preprocessing pour optimiser notre jeu de données : SMOTE, Tomek Links, PCA et RFE. Voici en quoi ils consistent et quel est leur intérêt :

— SMOTE (Synthetic Minority Over-sampling TEchnique) : concerne l'apprentissage supervisé, c'est-à-dire qu'il nécessite des données labellisées (fraude ou non). L'idée est que, dans un cas qui arrive souvent où dans nos labels il y a peu de données frauduleuses, alors l'apprentissage est peu efficace car il compare souvent des données non frauduleuses entre elles et les cas de fraude sont donc rares, donc plus enrichissants. Ainsi le but est de rééquilibrer les proportions artificiellement. Pour la détection de fraude, on a souvent moins de 0.1 % des données qui sont frauduleuses. L'idée est de plutôt s'approcher d'un ratio 99 % - 1% ou 95 % - 5%. Pour cela, on prend une paire de données frauduleuses et on ajoute à notre base de données une donnée frauduleuse qui se trouve à leur milieu dans le plan. On peut également choisir un barycentre différent ou le prendre de manière aléatoire. L'idée étant que notre nouveau point est proche de données frauduleuses, il est donc très suspect d'être frauduleux.

— Tomek Links : concerne également l'apprentissage supervisé mais agit à l'inverse de SMOTE pour supprimer quelques points du plan pour optimiser l'apprentissage. Dans un cas comme le nôtre, où il y a une dissymétrie brutale entre le nombre de labels fraude et non fraude, on cherche à préserver au mieux nos données les plus intéressantes qui sont celles ayant le label fraude. Ainsi on élimine du plan les points non fraude qui sont trop proches des points fraude. On dit qu'une paire de points du plan forme un Tomek Link si chacun est le plus proche voisin de l'autre. Si on a une paire fraude-non fraude qui forme un Tomek Link, on supprime le point non fraude. La surreprésentation des points non fraude fait que la perte n'est pas significative tandis qu'on protège l'information contenue dans les labels fraude. On peut ensuite tout à fait itérer ce procédé.

— PCA (Principal Component Analysis) : cet algorithme réduit le nombre de paramètres, c'est-à-dire la dimension du plan, et bascule dans un nouveau système de coordonnées pour optimiser l'accessibilité à l'information de la base de données. La trentaine de paramètres initiaux est superflue. On ne veut garder que les éléments les plus intéressants pour notre apprentissage et on dispose d'un critère simple de sélection. On calcule la matrice de covariance de nos paramètres. Puis on sélectionne le sous-espace propre associé aux plus grandes valeurs propres (plus grandes variances, donc avec le moins de redondance), ce qui est immédiat à faire après diagonalisation. Les vecteurs propres associés que nous prenons pour nouveaux paramètres sont orthonormés, ce qui garantit l'indépendance et la singularité des informations qu'on peut y exploiter.

— RFE (Recursive Feature Elimination) : cet algorithme tout comme le précédent vise à réduire le nombre de paramètres. On applique un algorithme de machine learning sur notre jeu de données pour faire notre apprentissage (par exemple l'arbre de décision) puis on sélectionne le paramètre ayant un pouvoir de décision le plus faible, c'est-à-dire celui qui est le moins retenu quand on décide de la nature (fraude ou non) d'un nouveau point. Ce paramètre est ensuite éliminé et on recommence notre apprentissage de

zéro. L'idée est d'itérer ce procédé pour finir avec un nombre restreint de paramètres (environ une dizaine).

**Ordre de la pipeline preprocessing** : l'ordre dans lequel on applique les algorithmes a son importance, ils ont été présentés dans l'ordre de la pipeline, voici pourquoi :

— SMOTE puis Tomek Links. Le SMOTE crée des données synthétiques entre les points existants. Le problème, c'est qu'il crée souvent des points bruités qui empiètent sur la classe majoritaire. En passant les Tomek Links après, on supprime les points qui sont trop proches les uns des autres entre les deux classes. Cela nettoie la frontière de décision que le SMOTE a rendue floue.

— PCA ensuite. La PCA cherche à maximiser la variance. Si on fait la PCA avant de rééquilibrer les classes, elle sera totalement biaisée par la classe majoritaire. En faisant la PCA après le rééquilibrage et le nettoyage, on s'assure que les composantes principales représentent bien la structure des deux classes de manière équitable.

— RFE après PCA. RFE est très coûteux car il nécessite d'entraîner l'algorithme à chaque fois. Ainsi on l'utilise pour affiner. Le seul bémol d'utiliser RFE après PCA, c'est qu'on perd l'interprétation heuristique qu'on peut faire sur l'importance d'un paramètre : l'œil humain ne peut plus lire des explications "naturelles".

Bien que la nécessité de faire du Preprocessing n'est plus à démontrer, il reste encore plusieurs indéterminations. Quels algorithmes faut-il mettre dans la pipeline ? Faut-il tous les prendre ou seulement certains parmi les 4 ? Ensuite, pour chaque algorithme gardé, il faut décider à quelle intensité on l'utilise en choisissant les paramètres comme le nombre de données frauduleuses à rajouter via SMOTE par exemple.

Enfin il est également possible de rendre ces algorithmes quantiques (par exemple QSMOTE [1]), c'est-à-dire agissant sur les vecteurs dans l'espace des états. Ou bien nous les utilisons pour optimiser notre base de données avant l'encodage quantique des vecteurs, ou bien nous les utilisons après l'encodage quantique, sur les vecteurs de notre FeatureMap. Le prétraitement post-mapping permet d'opérer directement sur la structure topologique de l'Espace de Hilbert, facilitant l'extraction de corrélations quantiques complexes qui resteraient cachées dans l'espace euclidien originel.

# 5 CLUSTERING

## 5.1 Clustering classique

## 5.2 Clustering quantique variationnel

Après avoir passé en revue les principales techniques de clustering quantique, nous avons retenu l'approche *variationnelle* (Variational Quantum Clustering, VQC) car c'est aujourd'hui la plus employée. Nous en reprenons directement la formulation proposée par Bermejo et Orús [7].

### • Encodage et états de référence

On considère un jeu de $N$ points $\mathcal{X} = \{\vec{x}_1, \ldots, \vec{x}_N\} \subset \mathbb{R}^d$ à partitionner en $k$ clusters. On associe à chaque point $\vec{x}_i$ un vecteur de paramètres $\vec{\theta}_i \in [0, 2\pi)^n$, où $n$ est le nombre de qubits dédiés à l'encodage. Dans sa forme

la plus simple, l'encodage consiste en une séquence de rotations $R_Y(\theta_{i,j})$ appliquées aux $n$ qubits initialement dans l'état $|0\rangle^{\otimes n}$ :

$$|\psi(\vec{\theta}_i)\rangle \;=\; \bigotimes_{j=1}^{n} R_Y(\theta_{i,j})\,|0\rangle \;=\; \bigotimes_{j=1}^{n}\Big(\cos\tfrac{\theta_{i,j}}{2}\,|0\rangle + \sin\tfrac{\theta_{i,j}}{2}\,|1\rangle\Big)\,.$$

Ce schéma d'*angle encoding* est le plus léger en profondeur, donc le mieux adapté aux contraintes NISQ [15]. Des encodages plus expressifs — *ZZ feature map*, *amplitude encoding*, ou ansatz entanglant paramétré par couches $U_{\text{ent}}(\vec{\theta}_i)$ — peuvent s'y substituer sans modifier le reste de l'algorithme.

À chaque cluster $c \in \{1,\dots,k\}$ est associé un *état de référence* $|\phi_c\rangle \in (\mathbb{C}^2)^{\otimes n}$, tiré aléatoirement au début de l'exécution et *gelé* pendant tout l'apprentissage. Les paramètres entraînables sont donc uniquement les $\{\vec{\theta}_i\}_{i=1}^{N}$, qui encodent la position de chaque point dans l'espace de Hilbert relativement aux ancres fixes $\{|\phi_c\rangle\}_{c=1}^{k}$. On notera que les $|\phi_c\rangle$ ne forment pas nécessairement une base orthonormée : ils sont tirés indépendamment et peuvent avoir des recouvrements non nuls entre eux, ce qui est sans incidence sur la validité de l'algorithme.

## • Matrice de fidélité et SWAP test

Le cœur du calcul est la *matrice de fidélité* $\mathcal{F} \in \mathbb{R}^{N\times k}$, définie par

$$\mathcal{F}_{ic} \;=\; \big|\langle\psi(\vec{\theta}_i)\,|\,\phi_c\rangle\big|^2.$$

On interprète $\mathcal{F}_{ic}$ comme un score continu d'appartenance du point $i$ au cluster $c$ : il vaut 1 si $|\psi(\vec{\theta}_i)\rangle = |\phi_c\rangle$ (à phase globale près), et 0 si les deux états sont orthogonaux. Contrairement à un $k$-means classique, qui produit une assignation dure, la formulation variationnelle manipule ici une *distribution* d'appartenance que l'on contraint ensuite à se concentrer sur un unique cluster via un terme de pénalité (cf. ci-dessous).

L'estimation effective de $\mathcal{F}_{ic}$ sur matériel quantique repose sur le *SWAP test*, qui extrait la fidélité entre deux états sans nécessiter de tomographie complète. On prépare un qubit ancillaire $A$ dans $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, on applique des portes contrôlées `CSWAP` entre $A$ et chaque paire de qubits correspondants de $|\psi(\vec{\theta}_i)\rangle$ et $|\phi_c\rangle$, puis on ramène $A$ dans la base $Z$ par une seconde Hadamard et on le mesure. On montre alors que la probabilité d'observer $A = 0$ vaut

$$\mathbb{P}(A=0) \;=\; \tfrac{1}{2}\Big(1 + \big|\langle\psi(\vec{\theta}_i)\,|\,\phi_c\rangle\big|^2\Big),$$

d'où l'estimateur sans biais $\widehat{\mathcal{F}}_{ic} = 2\hat{p}_0 - 1$, où $\hat{p}_0$ est la fréquence empirique observée sur $P$ répétitions. L'erreur quadratique de l'estimation décroît en $\mathcal{O}(1/P)$.

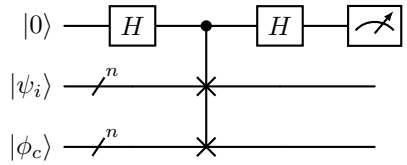


Figure 1 – Circuit du *SWAP test* estimant la fidélité $|\langle\psi_i|\phi_c\rangle|^2$. La ligne du haut est le qubit ancillaire ; les lignes inférieures, annotées $n$, représentent les deux registres de $n$ qubits portant respectivement l'encodage du point $|\psi_i\rangle = |\psi(\vec{\theta}_i)\rangle$ et l'état de référence $|\phi_c\rangle$. La porte `CSWAP` échange ces deux registres qubit à qubit, conditionnée à l'ancilla. La mesure finale de l'ancilla fournit un estimateur non biaisé de la fidélité.

## • Fonction de coût

Soit $D \in \mathbb{R}^{N\times N}$ la matrice des distances euclidiennes $D_{ij} = \|\vec{x}_i - \vec{x}_j\|_2$, calculée *une fois pour toutes* en prétraitement classique. La fonction de coût variationnelle s'écrit comme une combinaison linéaire de deux

termes :

$$\mathcal{C}(\vec{\theta}) = \underbrace{\sum_{c=1}^{k}\sum_{i,j=1}^{N} D_{ij}\,\mathcal{F}_{ic}\,\mathcal{F}_{jc}}_{\mathcal{C}_{\text{cluster}}(\vec{\theta})} + \lambda \underbrace{\sum_{i=1}^{N}\Big(\sum_{c=1}^{k}\mathcal{F}_{ic} - 1\Big)^2}_{\mathcal{C}_{\text{pénalité}}(\vec{\theta})}.$$

Le premier terme $\mathcal{C}_{\text{cluster}}$ pénalise l'appartenance simultanée de deux points *géométriquement éloignés* à un même cluster : il est d'autant plus grand que des paires $(i, j)$ à $D_{ij}$ élevée se voient attribuer des masses de fidélité $\mathcal{F}_{ic}\mathcal{F}_{jc}$ non négligeables sur un cluster commun. Sa minimisation encourage donc la compacité géométrique des groupes.

Le second terme $\mathcal{C}_{\text{pénalité}}$ impose que chaque point totalise une masse de fidélité proche de 1 sur l'ensemble des $k$ clusters, empêchant la solution dégénérée $\mathcal{F}_{ic} \to 0$ qui annulerait trivialement $\mathcal{C}_{\text{cluster}}$ sans produire de clustering. Il force ainsi, à la limite $\lambda \to \infty$, un comportement « one-hot » où chaque point se concentre sur un unique cluster.

Le scalaire $\lambda > 0$ règle l'équilibre entre ces deux contraintes et constitue l'unique hyperparamètre proprement variationnel du modèle. Un $\lambda$ trop faible autorise des assignations floues sans incitation à clusteriser nettement ; un $\lambda$ trop grand fait dominer la pénalité au point de masquer la géométrie sous-jacente. Les expériences du groupe précédent fixent $\lambda = 10$, valeur que nous retenons.

## • Algorithme complet

L'algorithme complet assemble les ingrédients précédents en une boucle classique–quantique (figure 2) :

1. **Initialisation.** Normaliser les données, calculer une fois pour toutes la matrice $D$, tirer aléatoirement $\{\vec{\theta}_i\}_{i=1}^{N} \in [0, 2\pi)^{n\cdot N}$ et $\{|\phi_c\rangle\}_{c=1}^{k}$.
2. **Évaluation quantique.** Pour chaque couple $(i, c)$, préparer $|\psi(\vec{\theta}_i)\rangle$ et $|\phi_c\rangle$, exécuter un SWAP test, et assembler la matrice $\mathcal{F}(\vec{\theta})$.
3. **Calcul classique du coût.** Évaluer $\mathcal{C}(\vec{\theta})$ à partir de $D$ et $\mathcal{F}$.
4. **Mise à jour.** Utiliser un optimiseur classique (L-BFGS-B avec gradients numériques dans notre implémentation) pour mettre à jour $\vec{\theta}$, puis répéter les étapes 2–4 jusqu'à convergence.
5. **Assignation finale.** Pour chaque point $i$, retourner le cluster $c_i = \arg\max_c \mathcal{F}_{ic}$.

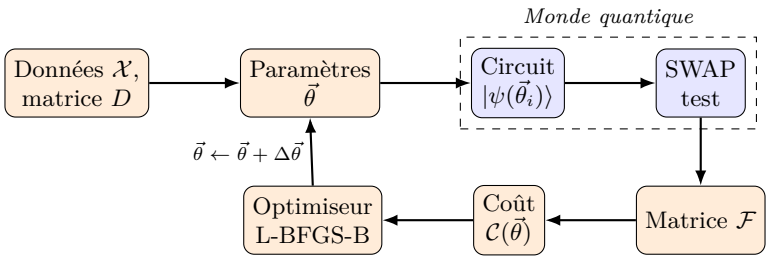


Figure 2 – Boucle d'apprentissage hybride du clustering variationnel quantique. Les blocs bleutés correspondent aux étapes exécutées sur simulateur/processeur quantique ; les blocs orange sont réalisés classiquement.

## • Choix du nombre de qubits et complexité

Le nombre $n$ de qubits dédiés à l'encodage joue un rôle crucial, mis empiriquement en évidence dans le rapport 2024-25. Deux régimes sont à éviter. En régime de *sous-paramétrisation* ($2^n < d$), la dimension de l'espace de Hilbert est trop faible pour distinguer les clusters et l'encodage projette des points distincts sur des états quasi confondus, induisant des fusions parasites. En régime de *sur-paramétrisation* ($2^n \gg d$), on effectue beaucoup de calculs inutiles et l'optimisation devient moins efficace. En pratique, choisir $n \approx \lceil \log_2 d \rceil$ offre le meilleur compromis.

La complexité par itération de l'optimiseur est en $\mathcal{O}(Nk \cdot P)$ exécutions élémentaires du circuit SWAP test, où $N$ est le nombre de points, $k$ le nombre de clusters, et $P$ le nombre de répétitions utilisées pour estimer chaque fidélité.

## 5.3 Implémentation

### • Métrique d'évaluation : l'indice de Rand ajusté (ARI)

Pour évaluer la qualité du clustering, nous utilisons l'*indice de Rand ajusté* (ARI, *Adjusted Rand Index*), qui mesure la similarité entre la partition prédite $\hat{\mathcal{P}} = \{\hat{C}_1, \dots, \hat{C}_k\}$ et la partition de référence $\mathcal{P}^\star = \{C_1^\star, \dots, C_{k'}^\star\}$. En notant $n_{ij} = |\hat{C}_i \cap C_j^\star|$, $a_i = \sum_j n_{ij}$ et $b_j = \sum_i n_{ij}$,

$$\mathrm{ARI} = \frac{\displaystyle\sum_{i,j} \binom{n_{ij}}{2} - \frac{1}{\binom{N}{2}} \sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\frac{1}{2}\left(\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right) - \frac{1}{\binom{N}{2}} \sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}.$$

Le numérateur compte les paires de points regroupées à l'identique dans les deux partitions (terme $\sum_{i,j} \binom{n_{ij}}{2}$), auquel on retranche le nombre attendu de telles paires sous tirage aléatoire. Ainsi, $\mathrm{ARI} = 1$ lorsque les deux partitions coïncident parfaitement et $\mathrm{ARI} = 0$ lorsque la partition prédite n'est pas meilleure qu'un tirage au hasard.

### • Première expérience

Nous avons implémenté l'algorithme VQC décrit plus haut en *Qiskit*. Le jeu de données synthétique est constitué de trois *blobs* gaussiens dans $\mathbb{R}^3$, chacun de cardinalité 20 (soit $N = 60$ points à partitionner en $k = 3$ clusters). Nous avons ensuite exécuté la boucle hybride décrite précédemment pendant 30 itérations de l'optimiseur, avec l'hyperparamètre $\lambda = 10$.

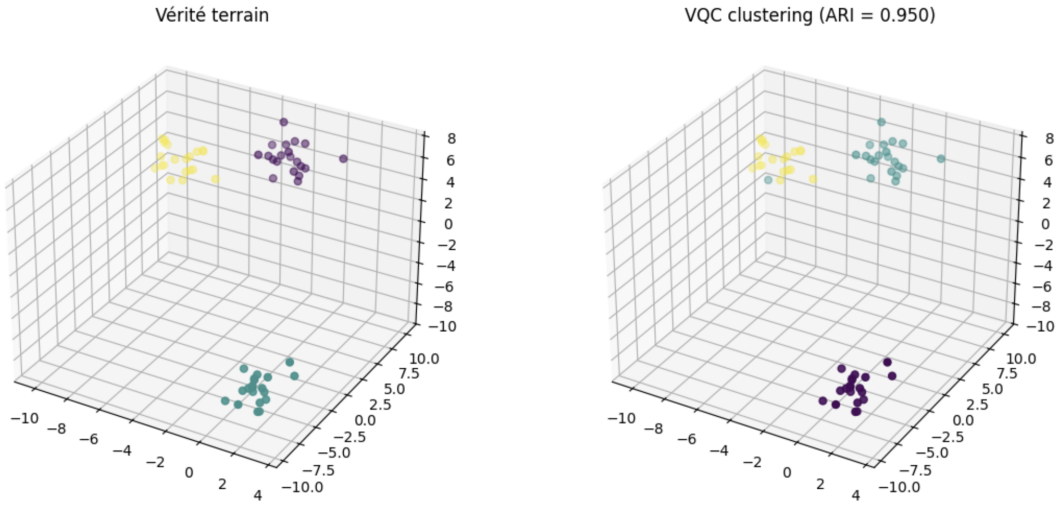


Figure 3 – Résultats de la première exécution du clustering variationnel quantique sur trois blobs gaussiens dans $\mathbb{R}^3$ ($N = 60$, $k = 3$, $\lambda = 10$, 30 itérations).

L'ARI obtenu sur cette exécution vaut 0,950, ce qui signifie que la VQC a bien compris la répartition en clusters. Ce résultat confirme que la boucle variationnelle parvient à exploiter la géométrie des blobs malgré le faible nombre d'itérations, et valide notre implémentation.

## 5.4 Rappels théoriques : SVM classique et astuce du noyau

Avant d'aborder sa variante quantique, il est indispensable de poser le cadre mathématique du Support Vector Machine (SVM) classique, car l'approche quantique vient s'y greffer directement sans en altérer la logique d'optimisation.

À l'origine, le SVM est un modèle de classification binaire cherchant à séparer linéairement des données réparties en deux classes [17]. L'idée directrice n'est pas seulement de trouver un hyperplan séparateur d'équation $\vec{w} \cdot \vec{x} + b = 0$, mais de trouver celui qui maximise la marge, c'est-à-dire la distance entre l'hyperplan et les points les plus proches de chaque classe.

### • Le problème primal et la marge souple

Mathématiquement, maximiser cette marge revient à minimiser la norme du vecteur normal à l'hyperplan, $\|\vec{w}\|$. Or, dans le cas de données réelles (et a fortiori dans la détection de fraude), les classes se chevauchent souvent à cause du bruit. Il est donc impossible de tracer une frontière parfaite. Pour y remédier, nous introduisons des variables réelles $\xi_i \geq 0$ qui autorisent certaines erreurs de classification moyennant une pénalité.

Le problème d'optimisation, dit « primal », s'écrit alors :

$$\min_{\vec{w},b,\vec{\xi}} \frac{1}{2}\|\vec{w}\|^2 + C\sum_{i=1}^{N} \xi_i$$

sous les contraintes $y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1 - \xi_i$. Le paramètre de régularisation $C$ contrôle le compromis entre la largeur de la marge (généralisation) et la pénalisation des erreurs sur le jeu d'entraînement. $y_i \in -1, 1$ est le label de du point $x_i$. $y_i(\vec{w} \cdot \vec{x}_i + b)$ est censé être supérieur ou égal à 1, ainsi $\xi_i$ est la variable de pénalisation de cette contrainte qu'on se permet d'assouplir.

### • Passage au problème dual

La formulation primale dépend directement de la dimensionnalité des données. Si nous voulons projeter nos données dans un espace très complexe pour capturer des comportements non linéaires, la taille de $\vec{w}$ explosera, rendant le calcul intraitable.

La solution canonique consiste à passer par la formulation duale de ce problème d'optimisation en introduisant les multiplicateurs de Lagrange $\alpha_i$. Après dérivation et annulation du gradient par rapport à $\vec{w}$ et $b$, le problème devient la maximisation d'une fonction quadratique :

$$\max_{\vec{\alpha}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j (\vec{x}_i \cdot \vec{x}_j)$$

sous les contraintes $0 \leq \alpha_i \leq C$ et $\sum_i \alpha_i y_i = 0$.

Ce problème présente deux propriétés fondamentales pour la suite de notre architecture :

1. **Parcimonie des solutions :** La majorité des multiplicateurs $\alpha_i$ s'annulent. Seuls les points d'entraînement situés sur la bordure de la marge ou du mauvais côté auront un $\alpha_i > 0$. Ces points critiques sont appelés les *vecteurs de support*. La frontière de décision finale ne dépend plus que d'eux.
2. **Dépendance exclusive au produit scalaires :** Les vecteurs de données $\vec{x}$ n'apparaissent dans la fonction à optimiser que sous la forme de produits scalaires $(\vec{x}_i \cdot \vec{x}_j)$, ce qui nous permet d'utiliser l'astuce du noyau ou kernel trick en anglais

### • Le Kernel Trick (Astuce du noyau)

Si il est trop optimiste de pouvoir tracer une frontière rectiligne pertinente dans l'espace de départ qui est de faible dimension, nous pouvons transformer nos données via une fonction non linéaire $\phi : \mathbb{R}^d \to \mathcal{F}$, où $\mathcal{F}$ est un espace de bien plus grande dimension.

Plutôt que de calculer explicitement les lourdes coordonnées de $\phi(\vec{x}_i)$ — ce qui serait coûteux voire impossible si $\mathcal{F}$ est de dimension infinie —, la littérature classique [18] souligne que l'algorithme dual n'a besoin que du produit scalaire dans ce nouvel espace. Il suffit alors de définir une fonction noyau $K$ telle que :

$$K(\vec{x}_i, \vec{x}_j) \;=\; \phi(\vec{x}_i) \cdot \phi(\vec{x}_j)$$

On remplace simplement le terme $(\vec{x}_i \cdot \vec{x}_j)$ par $K(\vec{x}_i, \vec{x}_j)$ dans le problème dual. L'algorithme se comporte alors comme si nous avions optimisé un hyperplan dans un espace très complexe celui où vivent les points $\phi(\vec{x}_i)$ , tout en conservant une complexité de calcul parfaitement maîtrisée. C'est précisément cette matrice des similarités $K$ (matrice de Gram) que l'ordinateur quantique va se charger de calculer pour nous dans l'approche hybride.

## 5.5 Quantum One-Class Support Vector Machine (Q-OCSVM)

Si le clustering variationnel vise à regrouper les données en plusieurs catégories, il est souvent plus naturel, dans le contexte de la fraude bancaire, de ne chercher qu'à modéliser ce qui est « normal ». La fraude étant par nature rare et multiforme, il est difficile d'en faire un cluster à part entière. Le *Quantum One-Class SVM* (Q-OCSVM) répond exactement à cette logique, en adaptant au monde quantique le concept d'estimation de support [19] : au lieu de partitionner l'espace, nous cherchons à tracer une frontière la plus ajustée possible autour des transactions légitimes. Le problème d'optimisation primal s'écrit ici :

$$\begin{aligned} \min_{\vec{w},\rho,\vec{\xi}} \quad & \frac{1}{2}\|\vec{w}\|^2 - \rho + \frac{1}{\nu N}\sum_{i=1}^{N} \xi_i \\ \text{sous contraintes} \quad & \vec{w} \cdot \phi(\vec{x}_i) \geq \rho - \xi_i, \quad \forall i \in \{1, \ldots, N\} \\ & \xi_i \geq 0, \quad \forall i \in \{1, \ldots, N\} \end{aligned} \tag{1}$$

Ainsi, bien que le SVM classique soit fondamentalement un algorithme d'apprentissage supervisé (nécessitant des étiquettes pour séparer deux classes distinctes), l'OCSVM s'inscrit dans un cadre strictement non supervisé. L'astuce géométrique consiste à fournir uniquement des données normales à l'algorithme (ou à défaut des données passées) et à contraindre l'optimisation pour que tous ces points soient confinés du même côté d'un hyperplan, tandis que l'origine de l'espace de représentation est artificiellement repoussée de l'autre côté, jouant le rôle de la classe adverse.

L'intérêt majeur de cette méthode pour notre pipeline hybride réside dans la séparation très nette des tâches. Nous n'utilisons le processeur quantique que pour une seule chose : évaluer à quel point deux transactions se ressemblent. Toute la partie d'optimisation (le tracé de la frontière) reste confiée à un algorithme classique.

### • Plongement quantique et matrice de Gram

Classiquement, les algorithmes comme le SVM utilisent l'astuce du noyau (*kernel trick*) pour projeter les données dans un espace de plus grande dimension (via un noyau gaussien RBF par exemple) afin de faciliter la séparation des points. Dans notre approche quantique, c'est le circuit lui-même qui sert de fonction de projection.

Concrètement, chaque transaction $\vec{x}_i \in \mathbb{R}^d$ est encodée sous la forme d'un état quantique à l'aide d'un circuit paramétré $U(\vec{x}_i)$. L'état obtenu est :

$$|\psi(\vec{x}_i)\rangle \;=\; U(\vec{x}_i)\,|0\rangle^{\otimes n}$$

L'espace dans lequel nous travaillons n'est plus un espace euclidien standard, mais l'espace de Hilbert de nos $n$ qubits, dont la dimension croît de manière exponentielle en $2^n$. En utilisant des circuits d'encodage qui intriquent les qubits (comme les feature maps basés sur des portes de Pauli [10]), nous exploitons des corrélations très complexes entre les différentes caractéristiques (montant, heure, localisation) de la transaction. La similarité entre deux transactions $\vec{x}_i$ et $\vec{x}_j$ correspond alors tout simplement à la probabilité de transition entre leurs deux états quantiques, c'est-à-dire leur fidélité :

$$K_Q(\vec{x}_i, \vec{x}_j) \;=\; \left|\langle\psi(\vec{x}_i)\,|\,\psi(\vec{x}_j)\rangle\right|^2$$

## • Le circuit de calcul (Compute-Uncompute)

Pour estimer cette fidélité sur machine, nous pourrions utiliser le *SWAP test* présenté dans la section précédente. Cependant, le SWAP test nécessite de doubler le nombre de qubits pour stocker les deux états simultanément, en plus d'un qubit ancillaire. Sur les processeurs actuels, chaque qubit supplémentaire apporte du bruit.

D'après la littérature d'apprentissage quantique à base de noyaux [20] il est préférable d'utiliser une technique appelée *inversion* ou *compute-uncompute*. L'idée est de n'utiliser qu'un seul registre de $n$ qubits. On applique d'abord le circuit d'encodage pour la première transaction, puis on applique l'inverse du circuit paramétré avec la seconde transaction :

$$|\psi_{i,j}\rangle \;=\; U^\dagger(\vec{x}_j)\, U(\vec{x}_i)\, |0\rangle^{\otimes n}$$

Si les deux transactions sont identiques ($\vec{x}_i = \vec{x}_j$), l'opération globale équivaut à l'identité, et le registre retourne exactement dans l'état $|0\rangle^{\otimes n}$. Si elles diffèrent, la probabilité de mesurer l'état de base diminue. Ainsi, en mesurant simplement la fréquence d'apparition de la chaîne de bits $00\ldots0$, nous obtenons une estimation directe de notre noyau quantique.

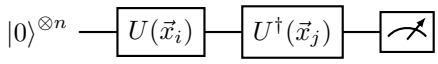


Figure 4 – Évaluation de la similarité entre deux transactions par la méthode d'inversion. Le registre est préparé selon les caractéristiques du point $\vec{x}_i$, puis subit la transformation inverse paramétrée par $\vec{x}_j$. La mesure de l'état fondamental $|0\ldots0\rangle$ nous donne directement la valeur du noyau $K_Q(\vec{x}_i, \vec{x}_j)$.

## • Entraînement et limites de passage à l'échelle

Une fois que le circuit quantique a calculé les similarités pour toutes les paires de transactions de notre jeu d'entraînement, nous obtenons une matrice dense, la matrice de Gram. Celle-ci est transmise à un optimiseur classique qui va chercher à isoler les données en ajustant un paramètre $\nu$ qui est inversement proportionnel au paramètre de pénalité $C$ indiqué dans l'équation d'optimisation du SVM dans la section précédente (et qui représente concrètement la proportion de faux positifs que la banque est prête à tolérer).

Le principal verrou de cette méthode, que nous avons dû affronter lors des tests sur les données de la BNP, est sa complexité quadratique. Pour $N$ transactions d'entraînement, il faut exécuter $N(N-1)/2$ circuits quantiques, et ce, pour plusieurs milliers de batchs à chaque fois afin d'avoir une estimation statistique fiable. Sur un jeu de données bancaire massif, le calcul complet de la matrice devient vite irréalisable en un temps raisonnable. C'est pourquoi nous avons dû accepter l'idée de ne conserver qu'un sous échantillon lors des implémentations de nos algorithmes sur le dataset fourni par la BNP.

## • Algorithme complet

L'implémentation de notre brique Q-OCSVM suit la séquence classique-quantique suivante :

1. **Préparation classique :** À partir de l'ensemble d'entraînement constitué de transactions supposées normales $\mathcal{X}_{\text{norm}} \subset \mathbb{R}^d$, on sélectionne (par sous-échantillonnage) un sous-ensemble gérable de $N$ transactions.
2. **Évaluation quantique (Construction de la matrice de Gram) :**
   — Initialiser une matrice $\mathcal{K}$ de taille $N \times N$.
   — Pour chaque paire de points $(\vec{x}_i, \vec{x}_j)$ avec $i \leq j$ :
     — Exécuter le circuit d'inversion $U^\dagger(\vec{x}_j)U(\vec{x}_i)\,|0\rangle^{\otimes n}$ sur le simulateur.
     — Estimer la fidélité $K_Q(\vec{x}_i, \vec{x}_j)$ via la fréquence de mesure de l'état $|0\rangle^{\otimes n}$ sur $P$ répétitions.
     — Remplir la matrice : $\mathcal{K}_{ij} = \mathcal{K}_{ji} = K_Q(\vec{x}_i, \vec{x}_j)$.
3. **Optimisation classique :** Fournir la matrice de Gram $\mathcal{K}$ et le paramètre $\nu$ à un solveur SVM classique (par exemple via `scikit-learn`) pour déterminer les vecteurs de support $\vec{\alpha}$ et le biais $\rho$.

4. **Inférence (Nouvelle transaction) :** Pour évaluer une nouvelle transaction $\vec{z}$ :
   — Utiliser le circuit quantique pour calculer les noyaux $K_Q(\vec{x}_{\text{sv}}, \vec{z})$ uniquement avec les transactions $\vec{x}_{\text{sv}}$ identifiées comme vecteurs de support lors de l'entraînement.
   — Calculer classiquement le score de décision $f(\vec{z}) = \sum_{\text{sv}} \alpha_{\text{sv}} K_Q(\vec{x}_{\text{sv}}, \vec{z}) - \rho$.
   — Classer $\vec{z}$ comme fraude si $f(\vec{z}) < 0$.

# 6 RÉSEAUX À COUCHES

# 7 ASSEMBLAGE D'UNE PIPELINE

## 7.1 Architecture modulaire

## 7.2 Résultats

# 8 TESTS ET COMPARAISONS

Comment évaluer notre modèle ? Dans cette partie nous testons notre pipeline en testant 4 modèles différents. Dans un second temps nous évaluons le troisième algorithme, celui de notre pipeline sur le jeu de données de la BNP.

## 8.1 Comparaison de 4 algorithmes sur un dataset tiers

1. Algorithme classique superivsé : Preprocessing + Random Forest (ou Tancrède)
2. Algorithme quantique non superivsé : Q-OCSVM
3. Algorithme hybride autosupervisé : Q-OCSVM + Random Forest
4. Algorithme quantique superivsé : Preprocessing + Q-SVM

Voici les résultats sur le dataset :

## 8.2 Application au jeu de données de la BNP

Pour évaluer notre pipeline directement sur le jeu de données de la BNP nous ne disposons malheuresemnt d'aucun label. Ainsi nous avons opté sur deux approches. Premièrement nous pouvons calculer le coefficient de silhouette de notre résultat. Ce coefficient est très répandu dans la littérature pour l'évaluation d'agorithmes non supervisés.

Le coefficient de silhouette est la moyenne des scores $s_i$ calculés en chaque point. On définit la séparation externe $b_i$ qui est la distance moyenne entre le point $i$ et tous les points du cluster voisin le plus proche ainsi

que la cohésion interne $a_i$ qui est la distance moyenne entre le point i et tous les autres points situés dans le même cluster.

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

A noter que chaque $s_i$ et donc le coefficient de silhouette sont dans l'intervalle $[0, 1]$. Le coefficient de silhouette est une métrique de validation qui permet de mesurer la qualité d'une partition de données en évaluant la cohérence de chaque point par rapport à son propre groupe (le cluster) face aux autres. Voir [**?**] et [**?**] pour plus de précisions sur le coefficient de silhouette et sa pertinence.

Voici le coefficient de silhouette que nous avons obtenu sur un échantillon de taille .... du jeu de données de la BNP + commentaires

Ensuite nous avons décidé de regarder en détail ce que notre modèle nous indiquait comme trade jugé frauduleux. Nous avons pris au hasard 50 données indiquées comme potentiellement frauduleuses par notre algorithme. Nous avons inspecté leurs features et il est aisé d'observer que cette échantillon est anormal dans une certaine mesure. Voici quelques exemples qui étaillent notre propos :

# 9 DIFFICULTÉS ET CRITIQUE

## 9.1 Limites matérielles

## 9.2 Absence de labels dans le jeu de données de la BNP

L'un des buts du projet fut de pouvoir évaluer notre modèle directement sur le jeu de données de la BNP. D'autant qu'un effort a été porté tout le long de l'année pour respecter les contraintes spécifiques de ce dernier. Comme expliqué dans l'introduction le jeu de données de la BNP qui a été fourni à notre groupe de PSC n'est pas labellisé. Cela pose bien sûr un problème pour l'entrainement d'un modèle car beaucoup sont basés sur l'apprentissage statistique des labels. Nous avons vu qu'il était possible de contourner cet obstacle par l'utilisation de modèles auto-supervisés c'est à dire en créant eux mêmes des "faux" labels pour faire de l'apprentissage sur ces derniers par la suite. De plus les labels servent également à évaluer un modèle. Ainsi un vrai casse tête fut de savoir comment évaluer la performance de nos modèles sur le jeu de données de la BNP. Nous avons finalement opter pour une double approche : calculer le coefficient de silhouette et analyser à la main les données catégorisées comme frauduleuses.

# 10 CONCLUSION ET IDÉES FUTURES

## 10.1 Bilan

## 10.2 Idées futures

# RÉFÉRENCES


[1] J. Thompson et al. 2025. A Quantum Approach to Synthetic Minority Oversampling Technique (SMOTE). Quantum Machine Intelligence.

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, *SMOTE : Synthetic Minority Over-sampling Technique*, Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[3] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, S. Lloyd, *Quantum Machine Learning*, Nature, vol. 549, pp. 195–202, 2017.

[4] I. Kerenidis, J. Landman, A. Luongo, A. Prakash, *q-means : A quantum algorithm for unsupervised machine learning*, Advances in Neural Information Processing Systems (NeurIPS), 2019.

[5] I. Kerenidis, J. Landman, *Quantum spectral clustering*, Physical Review A, vol. 103, 042415, 2021.

[6] A. Poggiali, A. Berti, A. Bernasconi, G. M. Del Corso, R. Guidotti, *Quantum Clustering with k-Means : a Hybrid Approach*, Theoretical Computer Science, 2024.

[7] P. Bermejo, R. Orús, *Variational quantum and quantum-inspired clustering*, Scientific Reports, vol. 13, 13284, 2023.

[8] M. Guo, H. Liu, Y. Li, W. Li, F. Gao, S. Qin, Q. Wen, *Quantum algorithms for anomaly detection using amplitude estimation*, Physica A, vol. 604, 127936, 2022.

[9] J.-M. Liang, S.-Q. Shen, M. Li, L. Li, *Quantum anomaly detection with density estimation and multivariate Gaussian distribution*, Physical Review A, vol. 99, 052310, 2019.

[10] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, J. M. Gambetta, *Supervised learning with quantum-enhanced feature spaces*, Nature, vol. 567, pp. 209–212, 2019.

[11] M. El Alami, N. Innan, M. Shafique, M. Bennai, *Comparative Performance Analysis of Quantum Machine Learning Architectures for Credit Card Fraud Detection*, arXiv :2412.19441, janvier 2026.

[12] N. Innan, A. Singh, M. Shafique, *CircuitHunt : Automated Quantum Circuit Screening for Superior Credit-Card Fraud Detection*, arXiv :2508.21366, août 2025.

[13] M. El Alami, A. Innan, N. Innan, M. Shafique, M. Bennai, *FiD-QAE : A Fidelity-Driven Quantum Autoencoder for Credit Card Fraud Detection*, arXiv :2512.12689, décembre 2025.

[14] R. Ubale, S. K. K., S. Deshpande, G. T. Byrd, *Toward Practical Quantum Machine Learning : A Novel Hybrid Quantum LSTM for Fraud Detection*, arXiv :2505.00137, avril 2025.

[15] F. M. Abbou, M. Bouhadda, L. Bouanane, M. Kettani, F. Abdi, A. Abid, *Comprehensive Analysis of VQC for Financial Fraud Detection : A Comparative Study of Quantum Encoding Techniques and Architectural Optimizations*, arXiv :2509.25245, septembre 2025.

[16] *Revolut Bank Transactions Fraud Detection Dataset*, jeu de données public, Kaggle. `[URL et auteur à compléter]`.

[17] C. Cortes, V. Vapnik, *Support-vector networks*, Machine Learning, vol. 20, pp. 273–297, 1995.

[18] B. E. Boser, I. M. Guyon, V. N. Vapnik, *A training algorithm for optimal margin classifiers*, Proceedings of the fifth annual workshop on Computational learning theory, pp. 144–152, 1992.

[19] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, R. C. Williamson, *Estimating the Support of a High-Dimensional Distribution*, Neural Computation, vol. 13, pp. 1443–1471, 2001.

[20] M. Schuld, N. Killoran, *Quantum Machine Learning in Feature Spaces*, Physical Review Letters, vol. 122, 040504, 2019.